

Odoo x LPU Hackathon | End-to-End Walkthrough

BEFORE RECORDING CHECKLIST

- [] Open <http://localhost:5173/> in browser (full screen or 1080p window)
- [] Open pgAdmin 4 (minimized, ready to alt-tab)
- [] Open VS Code with `server/server.ts` and `src/pages/DashboardPage.tsx` side by side (minimized)
- [] Microphone working
- [] Screen at 1920×1080
- [] Close all unnecessary apps/notifications
- [] Clear browser history/incognito so no autofill shows

SECTION 1 — INTRODUCTION (0:00 – 1:30)

[SHOW: Blank screen or a simple title card — "StockSense | Odoo x LPU Hackathon"]

SPEAK:

"Hi everyone. My name is Saikiran Lakotia, and this is StockSense — my submission for the Odoo x LPU Hackathon."

"Before I show you the application, let me quickly talk about the problem we were asked to solve."

1.1 The Problem Statement

[Keep title card on screen or show the PDF briefly]

SPEAK:

"The hackathon problem statement was about building an Inventory Management System — something that real businesses, especially warehouses and e-commerce companies, desperately need."

"The core pain points that were mentioned:"

"Number one — businesses have NO real-time visibility into how much stock they actually have. They're running on outdated spreadsheets or guesswork."

"Number two — when goods come in from a vendor or go out to a customer, there is no proper tracking. Items get lost, quantities mismatch, and nobody knows why."

"Number three — there is ZERO audit trail. If something goes wrong — a missing shipment, wrong quantity dispatched — you can't trace it back."

"Number four — there are no automated low-stock alerts. Products go out of stock and nobody finds out until a customer complains."

"Number five — everything is disconnected. The receipts team doesn't know what the dispatch team is doing."

SPEAK:

"So the challenge was — build a system that solves ALL of these problems, with a real database, real business logic, and a real working interface."

1.2 What I Built

[SHOW: Landing page at <http://localhost:5173/>]

SPEAK:

"What I built is StockSense — a full-stack, enterprise-grade inventory management system."

"It has a React frontend with TypeScript, an Express.js backend API, and a real PostgreSQL database at its core. No mock data. Every number you see is stored in actual database tables."

"Let me walk you through everything — from landing on the page for the first time, all the way through creating receipts, dispatching deliveries, and checking live stock."

SECTION 2 — LANDING PAGE TOUR (1:30 – 3:00)

[SHOW: <http://localhost:5173/> — full landing page]

SPEAK:

"This is the public landing page. The first thing a new user sees."

"You'll notice the sticky navigation bar at the top — the StockSense logo on the left, navigation links in the middle, and Sign In and Get Started buttons on the right."

"The hero section immediately communicates the value proposition — 'End Inventory Chaos. Master Every Stock Movement in Real Time.'"

"Below the headline, there's a description of what the system actually does — inbound receipts, outbound dispatches, warehouse history tracking."

"And these three trust indicators — PostgreSQL Real Persistence, Zero Mock Data, and Double-Entry Stock Logic — these are not marketing fluff. These are actual technical facts about how this system is built."

[SCROLL DOWN slowly]

"This browser mockup here is an actual screenshot of the live dashboard. Not a design mockup — a real screenshot taken from the running application."

[CONTINUE SCROLLING]

"The Features section explains the four core pillars — Real-Time Stock Tracking, Multi-Warehouse Support, Automated Low-Stock Alerts, and Full Audit Trail."

"The Workflow section shows the exact flow — from vendor receipt all the way to customer delivery."

"And the Architecture section shows what's happening under the hood — React talking to Express, Express talking to PostgreSQL."

SECTION 3 — AUTHENTICATION (3:00 – 5:30)

[CLICK: "Get Started" button — goes to /signup]

3.1 Sign Up

SPEAK:

"Let's create a new account to show the signup flow."

"I'll enter a name, a work email, and a password."

[TYPE: Name "Demo User", Email "demo@company.com", Password "demo1234"]

SPEAK:

"When I click Continue to Verification, the system does the following behind the scenes:"

"One — it sends the data to our Express API at the POST /api/signup endpoint."

"Two — the password is hashed using bcrypt with a salt factor of 10. The raw password is NEVER stored."

"Three — a 6-digit OTP code is generated and stored in the users table in PostgreSQL."

"Four — the OTP code is returned to the frontend for demo purposes — in production this would be emailed."

[CLICK: "Continue to Verification"]

SPEAK:

"Now we're on the OTP verification step. This is a demo environment so the code is shown right here in this banner."

[ENTER OTP CODE]

"When I verify, the server checks the OTP against what's stored in the database, and if it matches and hasn't expired, it issues a JWT token — a JSON Web Token — that the frontend stores in localStorage."

"Every API request from this point sends that token in the Authorization header. The server verifies it on every protected route."

[SHOW: Redirect to dashboard]

3.2 Sign In (existing user)

[NAVIGATE to /login — click Return to Home, then Sign In]

SPEAK:

"For existing users, the login flow is straightforward. Let me use the demo admin account."

[CLICK "Fill Demo Admin" button]

"The system auto-fills admin@stocksense.com and admin123 — the seeded administrator account. In a real deployment this would of course be removed."

[CLICK Sign In]

"Authenticated. JWT token issued. Let's get into the actual application."

SECTION 4 — DASHBOARD (5:30 – 8:30)

[SHOW: Dashboard at /app]

SPEAK:

"This is the Inventory Command Center — the main dashboard."

"Let's break it down from top to bottom."

4.1 Topbar

SPEAK:

"At the top, we have the application topbar. On the left — a hamburger menu that toggles the sidebar, the StockSense breadcrumb, and the current page name."

"On the right — the PostgreSQL Live indicator showing our database connection status in real time. Then the Dispatch and Receive Goods quick-action buttons, and a logout icon."

4.2 Sidebar

[POINT TO Sidebar]

SPEAK:

"The left sidebar is the main navigation. It's fully collapsible — I can click the Collapse button at the bottom to shrink it to icon-only mode, which saves screen space on smaller monitors."

[CLICK collapse, pause, then expand again]

"On mobile, it becomes a slide-in drawer — tap the hamburger icon and the sidebar slides in from the left."

"The navigation is organized into three sections — Workspace with the Overview dashboard, Operations with Receipts, Deliveries, and Stock Inventory, and Master Data with Products and Move History, plus Configuration for Warehouses and Racks."

4.3 KPI Cards

[POINT TO 4 KPI Cards]

SPEAK:

"These four KPI cards are the pulse of your warehouse operation."

"Receipts to Receive — how many inbound shipments are pending validation."

"Deliveries to Dispatch — how many outbound orders are ready or waiting."

"Low Stock Alerts — products that have fallen below their reorder threshold."

"Total Products — the size of your product catalog."

"These are NOT hardcoded numbers. They are live COUNT queries against the PostgreSQL database, refreshed every time you load the page."

"The logic is simple — for Receipts, it counts stock_moves where move_type = 'receipt' and status is 'waiting' or 'ready'. Same pattern for deliveries. Low stock counts products where on_hand minus reserved is less than or equal to the reorder point."

4.4 Recent Movements Table

[POINT TO table]

SPEAK:

"The Recent Movements table is a live audit trail — the last 8 stock movements across the warehouse."

"Each row shows the reference number — like WH/IN/0004 or WH/OUT/0003 — the type badge in color (green for RECEIPT, orange for DELIVERY), the route showing source to destination location, the contact (vendor or customer name), the scheduled date, and the current status."

"The color-coded status pills — DONE in green, WAITING in orange, READY in purple — make it immediately obvious what needs attention."

4.5 Warehouse Pulse Chart

[POINT TO Bar Chart]

SPEAK:

"On the right is the Warehouse Pulse — a bar chart showing stock movement activity by day of the week. It answers the question: when is our warehouse busiest?"

"This is built using pure CSS and JavaScript — no chart library dependency — making it lightweight and fast."

SECTION 5 — RECEIPTS: GOODS IN (8:30 – 12:00)

[CLICK: Receipts (In) in sidebar]

SPEAK:

"This is the Receipts page — everything that comes INTO the warehouse from vendors."

5.1 Receipts List View

SPEAK:

"You can see all existing receipts here. Each one has a reference number, the vendor it came from, the destination location inside the warehouse, the contact, the scheduled date, the status, and the number of line items."

"Notice the references — WH/IN/0001, WH/IN/0002, and so on. These are auto-generated by the system using a sequences table in PostgreSQL. Each warehouse and move type has its own counter. This is exactly how Odoo generates references in production."

[SHOW search bar]

"You can search by reference number or vendor name. The filter dropdown lets you narrow to specific statuses."

5.2 Create a New Receipt

[CLICK: + New Receipt button]

SPEAK:

"Let me create a new inbound receipt from scratch."

[FILL FORM: Select vendor, destination location, scheduled date]

"I select the vendor — this is the contact the goods are coming from."

"I select the destination location inside the warehouse — for example Rack A or WH Stock."

"I set the scheduled date."

[ADD LINE ITEM: select product, enter quantity]

"Now I add line items — which products are arriving and how many units."

"You can add multiple products to a single receipt. In real warehouses, a vendor shipment usually contains dozens of product types."

[CLICK: Save / Create]

"The receipt is created in WAITING status — this means it's scheduled but not yet validated. Stock has NOT been updated yet."

5.3 Validate the Receipt

[CLICK: Validate button on the receipt]

SPEAK:

"Now I click Validate. This is the most important button in the entire system."

"Here's what happens in the backend when Validate is clicked:"

"One — the server receives a PATCH request to `/api/receipts/:id/validate`."

"Two — the `InventoryService` checks the move is in a `validatable` state."

"Three — for each line item, it runs an UPSERT on the `stock_quants` table."

"Four — if this product already exists at this location, it ADDS the quantity to `on_hand_qty`."

"Five — if it doesn't exist yet, it creates a new `stock_quants` record."

"Six — the move status is set to `DONE` and `validated_at` is stamped with the current timestamp."

"Seven — all of this happens inside a single PostgreSQL transaction. If anything fails, the entire operation is rolled back. No partial updates."

[SHOW status changed to DONE]

"The receipt is now `DONE`. Let's verify the stock actually changed."

[CLICK: Stock Inventory in sidebar]

"Find the product we just received — and you can see the `on_hand_qty` has increased by exactly the quantity we entered. This is real database persistence."

SECTION 6 — DELIVERIES: GOODS OUT (12:00 – 15:30)

[CLICK: Deliveries (Out) in sidebar]

SPEAK:

"This is the Deliveries page — goods going OUT of the warehouse to customers."

6.1 How Stock Availability Works

SPEAK:

"This is where the system's business logic becomes really important."

"When you create a delivery, the system immediately checks if you have enough free stock to fulfill it."

"Free stock is calculated as: `on_hand_qty` minus `reserved_qty`. Reserved quantity is stock that's already committed to other pending orders."

"If free stock is greater than or equal to what the delivery needs — the status is `READY`. You can validate immediately."

"If free stock is NOT enough — the status is `WAITING`. You cannot validate until more stock arrives through a receipt."

"This prevents a critical real-world problem — dispatching goods you don't actually have."

6.2 Create a New Delivery

[CLICK: + New Delivery]

[FILL: Customer name, source location, product, quantity]

SPEAK:

"I'm creating a delivery for a customer. I select the source location — where the goods will be picked from. I add the product and quantity."

[SAVE]

"Watch the status. If the product has enough free stock, it will show READY. If not, WAITING."

6.3 Validate the Delivery

[CLICK: Validate on a READY delivery]

SPEAK:

"For a READY delivery, I click Validate."

"The backend logic here is different from a receipt. Instead of increasing stock, it DECREASES the on_hand_qty at the source location."

"Again — this is inside a PostgreSQL transaction. Atomic, consistent, and permanent."

[CHECK stock inventory to confirm decrease]

"Stock decreased. The movement is permanently recorded in the audit trail."

SECTION 7 — STOCK INVENTORY (15:30 – 18:00)

[CLICK: Stock Inventory in sidebar]

SPEAK:

"The Stock Inventory page gives you a real-time snapshot of every product across every location."

7.1 Stock Columns Explained

SPEAK:

"Three key columns:"

"ON HAND — the total physical quantity at this location."

"RESERVED — quantity committed to pending outbound deliveries."

"FREE TO USE — on_hand minus reserved. This is what's actually available to dispatch."

"This three-column model is the same model used by Odoo and SAP in enterprise deployments."

7.2 Low Stock Highlighting

SPEAK:

"Products where free-to-use quantity is at or below the reorder point are highlighted with a warning color. These are the products that need to be replenished immediately."

"The reorder point is set per product in the Products catalog. It could be 5 units, 50 units, whatever makes sense for that product's demand cycle."

7.3 Stock Adjustment

[CLICK: Adjust Qty on a product]

SPEAK:

"Sometimes there's a discrepancy between the system and what's physically on the shelf — items get damaged, miscounted, or misplaced."

"The Adjust Quantity function lets you enter the actual counted quantity. The system calculates the delta — the difference between what was recorded and what you counted — and updates the stock_quants record."

"This adjustment is also logged as a stock_move with move_type 'adjustment', so the audit trail captures it permanently."

SECTION 8 — PRODUCTS CATALOG (18:00 – 20:00)

[CLICK: Products in sidebar]

SPEAK:

"The Products page is your master catalog — every item that flows through the warehouse."

8.1 Product Details

SPEAK:

"Each product has:"

"— A SKU (Stock Keeping Unit) — a unique identifier like ELEC-001"

"— A name and category"

"— Unit of measure — units, kg, litres, boxes"

"— Cost per unit — used for inventory valuation"

"— Reorder point — the threshold that triggers a low-stock alert"

[CLICK: + New Product]

"Creating a product is straightforward. Fill in the details, set a reorder point based on how fast this product moves, and save."

"Once created, this product is available to add to receipts and deliveries immediately."

SECTION 9 — MOVE HISTORY (20:00 – 22:00)

[CLICK: Move History in sidebar]

SPEAK:

"This is the full, immutable audit trail. Every single stock movement that has ever happened in the system — from day one."

"Receipts, deliveries, adjustments, everything. With the reference number, type, route, contact, scheduled date, and final status."

"This is the accountability layer. If a manager asks 'who received this shipment?' or 'when was this item dispatched?', the answer is here."

"In traditional businesses, this information is either lost, in someone's email, or in a spreadsheet that nobody can find. With StockSense, it's one click away."

[SHOW search and filter]

"You can search by reference number, filter by move type, and sort by date."

SECTION 10 — WAREHOUSES & RACKS (22:00 – 23:30)

[CLICK: Warehouses & Racks in sidebar]

SPEAK:

"The Warehouses page is the configuration layer — where you set up the physical and virtual geography of your operations."

"Each warehouse has a name and a short code — like WH for the main warehouse."

"Under each warehouse, you have locations. Some are physical — Rack A, Rack B, Stock Room 1."

"Some are virtual — Vendor location is a virtual source for receipts, Customer location is a virtual destination for deliveries. This is how the system can use a single stock_moves table for all movement types."

"This exact model is borrowed from Odoo's own architecture — it's called the double-entry stock model, same as double-entry bookkeeping in accounting. Every movement has a source and a destination. This makes reversals, returns, and auditing trivial."

SECTION 11 — POSTGRESQL DATA (LIVE PROOF) (23:30 – 26:00)

[ALT-TAB to pgAdmin or open terminal]

SPEAK:

"Let me show you the actual database to prove everything you just saw is real persistence — not demo data in memory."

[OPEN pgAdmin → stocksense → Tables → stock_quants → View All Rows]

"Here is the stock_quants table — this is the live stock ledger. Every row represents a product at a specific location with its on_hand and reserved quantities."

"Every Validate click you saw updated rows in this table."

[SWITCH to stock_moves table]

"And here is stock_moves — every receipt and delivery we created. The reference numbers match exactly what we saw in the UI. The status column shows 'done' for everything we validated."

[OPTIONALLY show in psql]

"Or in the terminal:"

```
psql -U postgres -d stocksense
SELECT reference, move_type, contact, status FROM stock_moves ORDER BY created_at DESC;
```

"Real data. Real PostgreSQL. No mock. No in-memory fake."

SECTION 12 — TECHNICAL ARCHITECTURE (26:00 – 28:30)

[SHOW: VS Code with server.ts or split screen]

SPEAK:

"Let me quickly explain the architecture for anyone interested in the technical side."

Frontend

"The frontend is built in React 18 with TypeScript. Routing is handled manually using state — no React Router dependency — keeping the bundle small."

"Framer Motion handles all animations — the sidebar collapse, page transitions, and card entrances. All animations are under 300 milliseconds to feel premium without being flashy."

"The styling is pure CSS — no Tailwind, no Bootstrap. A custom design system with CSS variables for the dark purple theme, consistent spacing, and component-scoped class names."

Backend

"The backend is a Node.js Express server written in TypeScript, running on port 3001."

"Vite proxies all /api/* requests from port 5173 to port 3001, so in development you never deal with CORS issues and in production everything runs from one port."

"The InventoryService class contains all the business logic — stock validation, status computation, reference number generation."

Database Model

"The database has 8 tables. The key insight is the stock_moves table — it handles receipts, deliveries, internal transfers, and adjustments with a single move_type enum. This is the same design that Odoo uses."

"Every move has a source location and destination location. For a receipt from a vendor, source is the virtual Vendor location and destination is Rack A. For a delivery, source is Rack A and destination is the virtual Customer location. This makes the audit trail symmetric and complete."

"Stock is always stored in stock_quants — the live ledger. The moves table records what happened. The quants table records where things stand right now."

Auth

"Authentication uses bcryptjs for password hashing and jsonwebtoken for session management. OTP verification adds a second factor for account creation. JWTs are stateless — no session store needed."

SECTION 13 — FUTURE IMPROVEMENTS (28:30 – 30:00)

[SHOW: Dashboard or landing page — calm visual]

SPEAK:

"Given the 8-hour hackathon constraint, there are several areas I would expand in production:"

1. Email Integration

"Right now the OTP code is shown on screen for demo purposes. In production, this would be sent via a real email service like SendGrid or AWS SES. Users would never see the OTP on screen."

2. Role-Based Access Control

"The database already has a 'role' column — manager vs staff. The next step is enforcing those roles at the API level. A staff member shouldn't be able to validate deliveries or modify warehouse configuration."

3. Internal Transfers

"The schema already supports internal transfers — moving stock from Rack A to Rack B within the same warehouse. The move_type enum includes 'internal'. The UI for creating internal transfers was not built in the hackathon window."

4. Barcode Scanning

"In a real warehouse, operators scan barcodes. Integrating a barcode scanner API into the receipt and delivery forms would make the system usable without keyboard input on the warehouse floor."

5. Reporting & Export

"PDF and CSV export of move history, inventory valuation reports, and stock turnover analysis would be the next logical feature set."

6. Real-Time Updates

"Currently the dashboard refreshes on page load. Adding WebSocket support would push live updates to all connected dashboards — so when one operator validates a receipt, all other dashboards update automatically."

7. Multi-Company Support

"The schema could be extended with a company_id to support multiple businesses on the same platform — turning StockSense into a SaaS product."

SECTION 14 — CLOSING (30:00 – 31:00)

[SHOW: Landing page or dashboard]

SPEAK:

"To summarize what StockSense delivers:"

"A full-stack web application with a React frontend and Express backend."

"Real PostgreSQL database — every number is real, every action is persisted."

"Complete inventory workflow — from vendor receipt to customer delivery."

"Double-entry stock logic — same model as enterprise systems like Odoo and SAP."

"Full audit trail — every movement recorded permanently."

"Authentication with bcrypt and JWT — production-level security."

"A clean, premium dark-theme UI with smooth animations."

"The code is fully open source and available on GitHub at github.com/lakotiasaikiran/Odoo-StockSense."

"Thank you for watching. If you have any questions about the architecture, the database design, or any specific feature, feel free to reach out."

QUICK REFERENCE — VIDEO TIMESTAMPS

Time	Section
0:00 – 1:30	Introduction + Problem Statement
1:30 – 3:00	Landing Page Tour
3:00 – 5:30	Authentication (Signup + OTP + Login)
5:30 – 8:30	Dashboard (KPIs + Sidebar + Charts)
8:30 – 12:00	Receipts — Goods In (Create + Validate)
12:00 – 15:30	Deliveries — Goods Out (Stock check + Validate)
15:30 – 18:00	Stock Inventory (On-hand / Reserved / Free-to-use + Adjust)
18:00 – 20:00	Products Catalog
20:00 – 22:00	Move History & Audit Trail
22:00 – 23:30	Warehouses & Racks Configuration
23:30 – 26:00	Live PostgreSQL Data Proof (pgAdmin)
26:00 – 28:30	Technical Architecture Explanation
28:30 – 30:00	Future Improvements
30:00 – 31:00	Closing Summary

 SPEAKING TIPS

- Speak at a relaxed pace — slower than you think you need to
- Say what you are clicking BEFORE you click it
- Pause 1 second after navigating to a new page before speaking
- For technical sections, point to the screen as you explain
- Keep energy consistent — same tone from start to finish
- If you make a mistake — pause, take a breath, start the sentence again (easy to edit)

StockSense / Odoo x LPU Hackathon 2026 / github.com/lakotiasaikiran/Odoo-StockSense